

NH-7 Landslide Early Warning Pilot

Hackathon Build Plan — App + Website + Model/Backend

Scope: NH-7, Rishikesh – Karnaprayag – Joshimath (Char Dham corridor to Badrinath) | Version 1.0 | October 2026

1. Project Overview

A real-time landslide risk and early-warning system for the NH-7 corridor — the route that carries Char Dham Yatra traffic and daily commuters from Rishikesh through Karnaprayag to Joshimath and Badrinath. The system scores risk along the highway using terrain and live rainfall data, pushes alerts to travellers and officials, and lets people on the ground report what they're actually seeing. It ships as a mobile app and a website, with the road ahead being direct integration into a car's infotainment/navigation system.

Why NH-7, and why this framing

NH-7 already has an academic susceptibility map (Random Forest, Karnaprayag–Joshimath, ~97% success rate) — so don't pitch this as "first AI landslide prediction." Pitch it as the first **live, integrated, public-facing** system for this corridor: a static research map vs. something a pilgrim or driver actually opens before and during their trip. That distinction is true, defensible, and more impressive to judges than an unverifiable novelty claim.

2. Team Split & Workflow

Three tracks running in parallel from Day 1:

- **You** — the prediction model and the backend (data pipeline, API, alert logic, database).
- **Friend A / B** — the mobile app (citizen-facing: map, alerts, trip planner, field reports).
- **Friend A / B** — the website (public map + admin/DMA dashboard for validating reports and analytics).

The single most important thing to do on Day 1 is agree the **API contract** — the exact endpoints, request/response shapes — before any model is trained. Your teammates build the UI against a mocked version of your API immediately, instead of waiting on you. You swap the mock for the real model and data later without anyone else's code changing.

3. System Architecture (NH-7-scoped)

Data flow, kept deliberately simple for an 8-day build:

- **Data sources** → GSI Bhukosh / Bhuvan landslide points for Chamoli & Rudraprayag districts; DEM/slope/aspect via Google Earth Engine (SRTM or Bharat DEM); live rainfall via IMD or Open-Meteo API (free, no key needed); NH-7 road geometry from OpenStreetMap.
- **Backend** → fetches rainfall on a schedule, combines it with static terrain features per road segment, runs the model, stores a risk score per segment.
- **API** → one clean REST layer the app and website both call — they never touch the model or data pipeline directly.
- **App / Website** → render the map, handle subscriptions and alerts, collect field reports, show the admin dashboard.

- **Future** → the same API is what a car's infotainment system would call — voice and on-dashboard alerts, auto-suggested rerouting. Not built now; the architecture is just kept ready for it.

4. The Prediction Model — a realistic 8-day plan

Don't reach for a satellite-image deep-learning model for the hackathon — that's the research-paper track, on a 2-month timeline. For 8 days, a real, honestly-trained **Random Forest / XGBoost classifier** on tabular conditioning factors is achievable, credible, and squarely within scikit-learn, which is where your own ML learning already is. Real data, a real trained model, used honestly — that beats a flashier approach that's secretly faked.

Step by step

- Split NH-7 into fixed road segments (e.g. every 2–5 km) — each segment gets its own risk score.
- Pull a DEM for the corridor via Google Earth Engine; derive slope, aspect, and elevation per segment.
- Pull landslide points for Chamoli and Rudraprayag districts from Bhukosh/Bhuvan. If bulk access is slow or blocked, fall back to manually marking the known historical points along NH-7 from Bhuvan's landslide layer or published inventories — slower, but still real data, and it's a small corridor.
- Compute a couple of simple distance features (distance to river/drainage, distance to road) — straightforward in QGIS or with GeoPandas.
- Train a Random Forest classifier: landslide vs. non-landslide points as labels, terrain features as inputs. Hold out a spatially separate stretch of road for validation, not a random split.
- Wrap it in one inference function: live rainfall + static terrain features in, a risk score out, per segment, refreshed on a schedule.

If data acquisition eats too many days

Fall back to a transparent rule-based score (weighted slope + rainfall threshold) as a stated v1, with the ML model as a visibly in-progress upgrade. Say exactly that to the judges — a labelled, honest placeholder reads as more credible than a model you can't explain under questioning.

5. Backend — stack & API contract

- **Framework:** FastAPI (Python) — fast to stand up, you already know Python, auto-generates docs your teammates can read without asking you.
- **Storage:** SQLite for the hackathon — zero setup. Swap for Postgres later if the project continues.
- **Hosting:** Render or Railway free tier, or run locally with ngrok for the live demo as a backup.

Minimum endpoints to define on Day 1

Endpoint	Purpose
GET /risk-map	All NH-7 segments with current risk level — powers the main map.
GET /route-risk?from&to;&date;	Trip-planner: risk along a specific stretch for a travel date.
POST /subscribe	Register a user/location for alerts (push, SMS).
GET /alerts	Active alerts for a subscribed user.

Endpoint	Purpose
POST /field-report	Citizen submits a geotagged photo + description.
POST /admin/validate-report	Admin accepts/rejects a pending field report.
GET /history	Past recorded landslide events on the corridor, for the map.

6. Feature List — built to be judged

Grouped so you know what's load-bearing vs. what earns extra points vs. what's vision only.

Core (must have — the demo doesn't work without these)

- Live, colour-coded NH-7 risk map, segment by segment.
- Risk scores that actually move with live rainfall data — not static.
- Push/SMS alerts for subscribed travellers on a given stretch.
- **Trip / Yatra planner** — enter travel dates and a stretch of NH-7, get a risk readout before you leave. This is the single feature most worth polishing: it's the one a judge immediately understands the value of, because it's exactly what a Char Dham traveller would actually want.
- Citizen field-report submission (geotagged photo + note).
- Admin dashboard to validate reports and see basic analytics.
- Historical landslide markers on the map, so the risk map isn't presented as coming from nowhere.

Differentiators (strong effort-to-impact ratio)

- **WhatsApp bot** for risk checks and alerts — huge reach in India, works for pilgrims who won't install a new app.
- Hindi + English UI at minimum.
- **Offline/low-connectivity fallback** (SMS keyword check) — genuinely necessary in these mountains, and judges who know the terrain will register that you thought about it.
- **Explainable risk** — show the top factors behind a segment's score (steep slope, recent rainfall spike, past history). Cheap to add once you have feature importances, and builds trust.
- **Voice alerts** (text-to-speech) — doubles as a live preview of the infotainment future-scope.
- QR codes at dhabas/checkpoints along the route for a quick scan-and-check — a nice physical-world touch for the demo video.
- Frame the citizen-report loop explicitly as a flywheel: “every verified report makes the model better.” True, and a story judges respond to.

Future scope (say it, don't build it)

- Direct in-car infotainment integration — voice and dashboard alerts, auto-suggested rerouting.
- Scale to all four Char Dham routes, then statewide.
- Partner integration with the Border Roads Organisation, NHAI, and the State Disaster Management Authority.
- Upgrade the core model to satellite-image deep learning segmentation — this is explicitly the research-paper track already underway, which gives the pitch a credible “what's next” instead of a vague one.

7. Day-by-Day Build Plan

Day	Focus
1	Lock scope + full API contract. Start Bhukosh/Bhuvan data-access check immediately (it's the slowest unknown). Teammates start wireframes against the mocked API.
2	Pull DEM for the corridor, derive slope/aspect/elevation (Earth Engine). Acquire landslide points for Chamoli/Rudraprayag. Stand up FastAPI skeleton + SQLite schema.
3	Feature engineering (join points to terrain + rainfall). Start RF/XGBoost training. /risk-map live with early/placeholder scores so the frontend has something real to render.
4	Finish model training + spatial-holdout validation. Wire the real model into the backend. Teammates integrate the live map against the real API.
5	Build the trip-planner endpoint + alert/subscription logic. Teammates build the planner UI and notification handling.
6	Build field-report + admin-validation endpoints. Teammates build the report form and admin dashboard.
7	Add differentiators as time allows (WhatsApp bot, voice alerts, multilingual, explainability). Full end-to-end integration testing.
8	Polish UI, rehearse the live demo, record a backup demo video, prep the pitch deck and Q&A; answers (including the "why isn't this a deep-learning model" question — you have a good, honest answer for that now).

8. Pitch Tips

- Open with the human stakes, not the tech — Char Dham pilgrim volume, the 2025 Uttarakhand flash flood, then show the live map within the first minute.
- Say plainly that the model uses real GSI/satellite/rainfall data and a real trained classifier — name the sources. Specificity reads as credibility.
- Present the deep-learning upgrade and the research paper as the stated next phase, not something you're pretending already exists — a visible, credible roadmap is a strength, not an admission of weakness.
- Demo the trip planner live if you demo nothing else — it's the feature a judge can personally feel the value of in five seconds.